Kaggle Agent

Automatically run your training scripts on Kaggle (free GPU) and get the results back. Drop a Python script in, and the agent pushes it to Kaggle, waits for it to finish, and downloads the outputs.

Files

File	Purpose
`kaggle_agent.py`	The runner (CLI, Python API, and watch mode)
`example_train.py`	Small PyTorch script to test the pipeline
`README.md`	This file

How it works

```
your script  ->  kaggle_agent.py  ->  Kaggle (private notebook, GPU)
                       |                          |
                       |<---- status polling ----|
                       v
              results/<job-name>/  (models, logs, csv)
```

1. Your script is wrapped in a `kernel-metadata.json` and pushed with the Kaggle CLI.
2. The agent polls the status every 20 seconds (queued, running, complete, error).
3. When finished, everything saved in `/kaggle/working` is downloaded to `results/<job-name>/`.

Setup (one time)

1. Install the Kaggle CLI:

bash

```
pip install kaggle
```

2. Create an API token: Kaggle -> Settings -> **Create New Token**. This downloads `kaggle.json`.
3. Put it in place:

bash

```
mkdir -p ~/.kaggle
mv kaggle.json ~/.kaggle/
chmod 600 ~/.kaggle/kaggle.json
```

On Windows, put it in `C:\Users\<you>\.kaggle\`. Alternatively, set the environment variables `KAGGLE_USERNAME` and `KAGGLE_KEY`. 4. Verify your phone number on Kaggle (Settings -> Phone verification). GPU and internet access are disabled on unverified accounts.

Test your setup:

bash

```
kaggle kernels list --mine
```

Usage

Run one job (push, wait, download)

bash

```
python kaggle_agent.py run example_train.py --gpu
```

Other commands

bash

```
python kaggle_agent.py push   train.py --gpu     # push only
python kaggle_agent.py status my-job-name        # check status
python kaggle_agent.py output my-job-name        # download results
```

Options

Flag	Meaning
`--name NAME`	Job name (defaults to the script filename)
`--gpu`	Enable GPU
`--no-internet`	Disable internet (needed for some competitions)
`--dataset user/name`	Attach a dataset (repeatable)
`--competition slug`	Attach competition data (repeatable)

Fully automatic mode (watch folder)

```bash
python kaggle_agent.py watch
```

Leave this running. Drop a script into `jobs/` and it runs by itself:

```
jobs/
  mymodel.py        <- your training script
  mymodel.json      <- optional settings
  done/             <- successful jobs are moved here
  failed/           <- failed jobs are moved here
results/
  mymodel/          <- downloaded outputs
```

Example `mymodel.json`:

```json
{
  "gpu": true,
  "internet": true,
  "datasets": ["username/my-dataset"],
  "competitions": ["titanic"]
}
```

Use from Python or from an AI agent

```python
from kaggle_agent import run

status, results_dir = run("train.py", gpu=True)
print(status, results_dir)
```

For an AI agent, the simplest integration is: the agent writes a `.py` file into `jobs/`, then reads `results/<name>/metrics.json` when it appears and decides what to try next.

Rules for your training scripts

- **Save outputs to `/kaggle/working/`.** Anything saved elsewhere is lost when the session ends.
- **Competition and dataset files** are mounted read-only at `/kaggle/input/<name>/`.
- **Use a fallback path** so the script also runs locally:

```python
OUT = "/kaggle/working" if os.path.exists("/kaggle/working") else "."
```

- **Install extra packages** at the top of the script if needed (requires internet enabled):

```python
import subprocess
subprocess.run(["pip", "install", "-q", "some-package"])
```

- Print progress with `print()`. Logs are downloaded with the results.

Kaggle limits to know

- Free GPU quota is about 30 hours per week (check your quota on your Kaggle profile).
- A single session runs for at most about 9 to 12 hours.
- Disk space in `/kaggle/working` is limited (about 20 GB).
- Notebooks are created as **private** by default.

Troubleshooting

Problem	Fix
`401 Unauthorized`	Re-download `kaggle.json`, check it is in `~/.kaggle/`
`kaggle: command not found`	Run `pip install kaggle`, make sure the pip scripts folder is on your PATH
GPU or internet not working	Verify your phone number in Kaggle settings
`403 Forbidden` on competition data	Join the competition and accept its rules on the Kaggle website first
Status stuck on `queued`	Kaggle's GPU queue can be busy, wait or try again later
Status `error`	Open `https://www.kaggle.com/code/<user>/<job-name>` and read the log, or check `results/<job-name>/`
Job name conflict	Use a unique `--name`, pushing the same name updates the existing notebook

Tips

- Test your script on a tiny sample (few epochs) first, so you do not waste GPU hours on a bug.
- Write a `metrics.json` at the end of every run. It makes it easy for an agent (or you) to compare runs.
- Pushing the same job name again creates a new version of the same notebook.